



Towards A Standardized Aerial Platform: ICUAS'22 Firefighting Competition

Lovro Markovic¹ · Frano Petric¹ · Antun Ivanovic¹ · Jurica Goricanec¹ · Marko Car¹ · Matko Orsag¹ · Stjepan Bogdan¹

Received: 1 December 2022 / Accepted: 4 June 2023 / Published online: 10 July 2023
© The Author(s) 2023

Abstract

This paper presents an Unmanned Aerial Vehicle (UAV) platform used for the competition held at the International Conference for Unmanned Aircraft Systems 2022 (ICUAS'22). The envisioned scenario in the competition is delivering an extinguishing ball to a fire with a UAV. Driven on that thought, the qualifying round of the competition is organized in a realistic simulation environment. The competitors are required to navigate through a cluttered environment into a free space, where they perform a search for the target and deliver the ball as close as possible to that target. The competition finals were organized as a part of the conference, with the same goal as in the simulation qualifiers. The UAVs used for the competition are prepared and provided by the organizers. A detailed description of the UAV software and hardware is outlined in the paper as a step towards standardizing aerial research platform.

Keywords Unmanned Aerial Vehicle (UAV) · Standardized multirotor platform · Competition · Autonomous aerial vehicles

1 Introduction

Robotics competitions exist in different shapes and forms and have been growing more and more popular every year.

Lovro Markovic, Frano Petric, Antun Ivanovic, Jurica Goricanec, Marko Car, Matko Orsag and Stjepan Bogdan contributed equally to this work.

✉ Lovro Markovic
lovro.markovic@fer.hr

Frano Petric
frano.petric@fer.hr

Antun Ivanovic
antun.ivanovic@fer.hr

Jurica Goricanec
jurica.goricanec@fer.hr

Marko Car
marko.car@fer.hr

Matko Orsag
matko.orsag@fer.hr

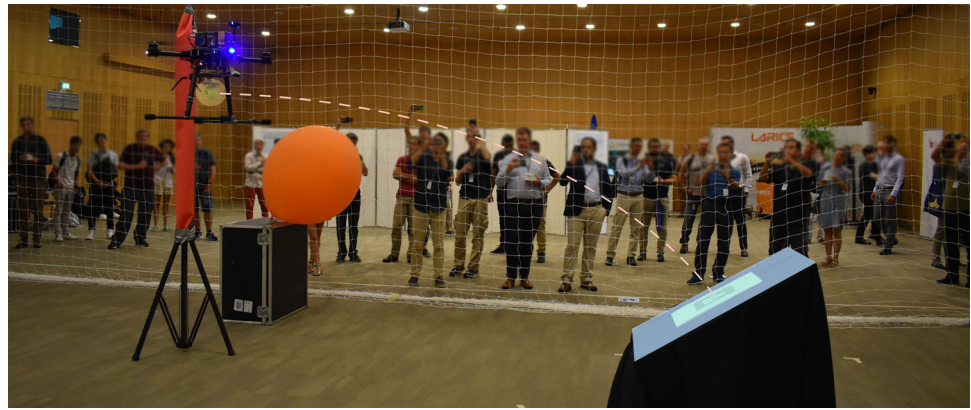
Stjepan Bogdan
stjepan.bogdan@fer.hr

¹ Faculty of Electrical Engineering and Computing, University of Zagreb, Unska 3, Zagreb 10000, Croatia

From the humble beginnings in 1970s to recent multi-million dollar competitions, they were used to showcase state-of-the-art technology, develop new materials and algorithms, engage the general public and promote STEM education and research. More recently, robotics competitions have been recognized as drivers towards repeatability and reproducibility of research within robotics community, and multiple instances have been organized to bridge the gaps between competition scenarios and scientific experiments. One such initiative was Robot Competitions Kick Innovation in Cognitive Systems and Robotics (RoCKIn) [2] where, in contrast to previous competitions (such as early RoboCup [3] competitions), the performance of the robotic system was not assessed by comparing the results between teams on high-level tasks, rather several benchmarks were introduced that evaluated the performance of the robots in more detail. Similar approach was undertaken in European Robotics Challenges [4], and was later further advanced through European Robotics League (ERL) which was built on top of euRathlon [5]. Currently, the METRICS project [6] is at the forefront of the robotics competitions based on functional and task benchmarks.

Nowadays, the most famous robotics competitions (DARPA [7], ERL [8], MBZIRC [9]) are multi-domain competitions aimed towards well-equipped research laboratories and

Fig. 1 The flight arena at ICUAS 2022 held in Dubrovnik, Croatia. A large audience gathers as the UAV navigates the obstacles to find the target. The UAV is captured just before launching the ball towards the target on the right. Obstacles in the arena are composed of balloons and poles made of soft material to minimize the damage in case of a crash. The safety net around the flight arena ensures safety for the observers. The YouTube stream of the entire competition is available in [1]



usually require a large team to develop multiple custom robotic platforms over relatively long periods of time. The requirement of having to buy or develop their own platforms and commit multiple years towards development poses as a high entry barrier to newcomers and smaller, more focused research groups. Having participated in both EuRoC, MBZIRC and ERL, we have identified a niche which could bring aerial robotics competitions to a larger audience. This niche consists of sprint-like competitions that do not require large teams, take a couple of months to complete and use a standardized open-source aerial robot platform. Such competition could lower the barrier to entry and engage more research groups, especially undergraduate students, if the following conditions are met:

- The competition is highly visible and can reach a large number of research groups
- Most of development for one instance of the competition can be performed in a realistic simulation environment
- Software stack for the robot enables seamless transition from simulation to the real platform
- Aerial platform is modular and can be extended with various sensors and actuators
- Aerial platform is affordable and available
- The scoring is transparent and the results are reproducible.

Based on the Aerial robotics control and perception challenge we organized at Mediterranean Conference on Control and Automation in 2018 [10], we believe that tying a competition to an aerial robotics conference can bring the competition into spotlight and expose it to a large number of research groups. In addition to increased visibility, having a competition collocated and as a part of a conference such as ICUAS provides the competitors with ability to meet not just other competitors but also researchers in the field (as evidenced by Fig. 1), further increasing attractiveness of the competition.

With the COVID-19 pandemic still ongoing with various degrees of severity in different parts of the world, the

uncertainty of travel imposes a significant constraint on the competitions. Various competitions employed different strategies for remote competitions: a) fully virtual competitions using real world data [11]; b) competitions in which the teams send their equipment to the location but participate remotely [12], c) fully remote participation on a robot provided by the competition organizers [13], d) hybrid events in which teams are allowed to participate by either being on site or performing tasks in their own laboratory remotely [14]. For the ICUAS'22 UAV Competition, we adopted an approach similar to [13] by providing the UAV platform that the teams will use in the finals at the conference venue and concentrating on seamless transition from simulation to the real world. This enables teams to rapidly deploy their solutions on-site, but also facilitates remote participation if required. The use of a standard platform is not new in robotics competitions, as RoboCup Standard Platform League¹ is one of the oldest still running competitions. More recently, RoboCup@Home [15] also adopted two standard platforms, and we believe similar can be beneficial for aerial robotic competitions by leveling the playing field.

In this work, we describe the development of a software stack for an aerial robot that would satisfy aforementioned conditions which ensure lower barrier to entry and higher engagement in aerial robotics competitions. The aerial platform presented in this paper integrates components from multiple research projects conducted in the Laboratory for Robotics and Intelligent Control Systems at the University of Zagreb Faculty of Electrical Engineering and Computing, coupled with the best practices from other research laboratories such as [16–18]. A number of components described herein are beyond the scope of the ICUAS'22 UAV Competition, but open up the possibility of different scenarios for future competitions and increase the overall attractiveness of the platform.

¹ RoboCup is an international initiative that fosters research in robotics and artificial intelligence, on multi-robot systems in particular - <https://spl.robocup.org/>

Following the introductory section, Section 2 describes the aforementioned UAV platform, both in terms of hardware and software. Section 3 showcases the deployment of developed platform in the ICUAS'22 UAV Competition, while Section 4 discusses the reach of the ICUAS'22 UAV Competition. Section 5 brings forth concluding remarks and briefly discusses the future of ICUAS UAV Competitions.

2 UAV Platform

Our hypothesis is that a standardized platform serves as an enabler of successful competition, allowing the teams to participate with little or no additional cost. It also serves to level the playground for the competitors, allowing them to focus on the control and software side of the problem. In this section, we describe this proposed UAV platform. First, the hardware components of a general-purpose UAV platform for research and development are outlined. In addition, the software modules that enable the execution of autonomous missions in a complex environment are described. Finally, the paper disseminates how all the components are integrated within the Gazebo software-in-the-loop (SITL) environment to facilitate an easier transition from the simulator to the real world.

2.1 Hardware Components

Although there are many off-the-shelf UAVs available, their hardware is usually tightly integrated and closed for modifications. One of the goals of this paper is to provide a standardized list of "must-have" hardware components of an open aerial platform capable of autonomous navigation.

Frame and Propulsion The most basic component of the aerial vehicle is its frame. It must be strong enough to withstand crashes, but also light enough to ensure longer flight times. Furthermore, the propulsion system is another integral element of aerial vehicles. A general rule of thumb is that the drone should reach hover at 35% thrust and maximum payload weight. Once this is achieved, adding more propellers only increases battery consumption, providing little additional benefit other than an increase in fault tolerance. The vehicle presented in this paper has a carbon fiber frame with four arms arranged in a cross structure. The propulsion system consists of four 15-inch T-Motor 1503 propellers and T-Motor MN4014 KV400 motors, resulting in a hover state at 38% of maximum thrust. The system is powered with 12000mAh 25V battery, which provides up to 20 minutes of flight time.

It is important to note that these components must be integrated in a highly modularized manner, as they typically bear the brunt of the impact during a crash and should therefore be easily replaceable.

The Autopilot The main job of the autopilot is to achieve stable vehicle flight by integrating all of the major vehicle components such as motors, battery, RC receiver, and optional sensors. Among the necessary functions that such a hardware piece should have are attitude (rate) control and state estimation using available integrated sensors such as an inertial measurement unit (IMU) and barometers. The final and perhaps most important requirement is that the autopilot is able to communicate with MAVLink² messages. This feature enables the UAV platform to communicate via the Mavros³ interface, which is essential for smooth integration with any state-of-the-art autonomy package based on the Robot Operating System (ROS) framework.

The UAV platform presented in Fig. 3 uses a *Cube Orange* autopilot hardware placed on a ProfiCNC/HEXKore high-current power board. It runs the Ardupilot - Copter⁴ firmware, a full-featured, open-source UAV controller enabling flight in various control modes, mission planning, sensor integration, etc. Although it has a large feature set, the proposed UAV platform uses only one - attitude (rate) and altitude control. This further relaxes hardware constraints and allows other autopilot systems such as PX4, BetaFlight, etc. to be used interchangeably.

The Onboard Processing Computer The onboard processing computer is an integral hardware piece, on which the software modules shown in Fig. 2 are executed. Starting from the left, there is a mapping module that provides information about the occupancy state of the environment. A global path planner uses this information to generate a collision-free path through the desired waypoints. The trajectory generator's task is to provide feasible trajectory points based on the given path and given dynamic constraints. The state estimation module provides the complete UAV body state, both translation and rotation, based on the available sensor data such as IMU measurements, visual SLAM pose, LiDAR SLAM pose, etc. Finally, the onboard controller is responsible for tracking the generated trajectory by providing attitude (rate) and thrust commands to the UAV, i.e., the low-level attitude controller running on the Ardupilot firmware. The transmission of incoming control inputs and outgoing sensor data to and from the UAV system is handled by the Mavros package. The implementation of the specified pipeline is presented in this paper and is described in the corresponding *uav_ros_stack*[19] ROS packages.

Such a modular framework is capable of performing autonomous missions using a diverse sensor suite. As

² MAVLink is the standardized common message set for micro aerial vehicles - <https://mavlink.io/en/messages/common.html>

³ MAVROS is a MAVLink extendable communication node for ROS - <http://wiki.ros.org/mavros>

⁴ Ardupilot (Copter): A full-featured, open-source multicopter controller - <https://ardupilot.org/copter/>

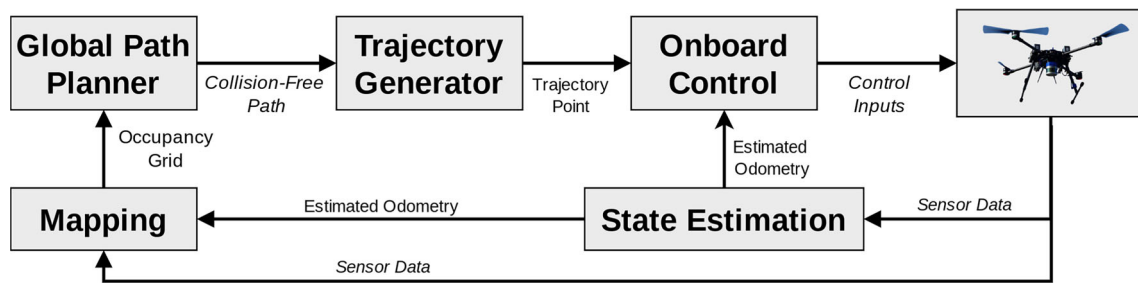


Fig. 2 This figure shows a complete diagram of the various software modules running the computer onboard the UAV. The connection between modules is achieved via the Robot Operating System (ROS) middleware framework, running on the Ubuntu Linux 20.04 operating system

explained earlier, the importance of the Mavros package is reiterated here. It is the bond that connects the low-level autopilot component to the on-board computer. All data sent to and received from the autopilot needs to go through the Mavros interface. This includes the attitude (rate) and thrust control inputs, sensor data such as GPS and IMU and RC input values among others.

An Intel NUC, with the Ubuntu Linux 20.04 operating system running ROS Noetic, is used as the onboard computer on the presented UAV platform (Fig. 3).

The Peripherals Peripherals include anything that is not part of the vehicle's basic functions but adds some value to the overall system. Their utility may take the form of enhanced navigation or environmental awareness, such as LiDAR or visual SLAM, or the ability to perform a unique task, such as payload delivery, aerial manipulation, visual inspection, etc. It is important to note that the frame of the vehicle should be designed to allow the mounting of additional peripherals.

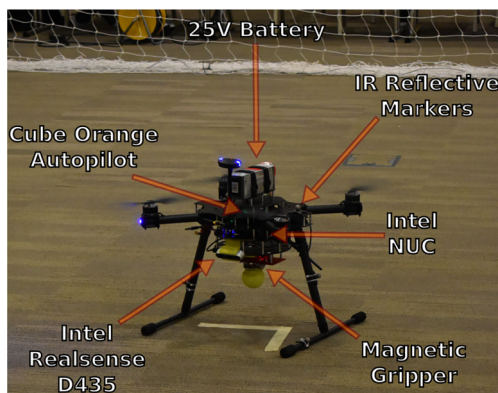


Fig. 3 This figure shows the UAV platform used in the competition along with its relevant components. A 25V battery provides a flight time of up to 20 minutes. To enable autonomous navigation, the Intel NUC is tightly integrated with the Cube Orange autopilot in the core of the UAV body. Reflective markers combined with the Optitrack motion capture system provide precise and stable localization, while the Intel Realsense D435 camera provides environment sensing capabilities. Finally, the magnetic gripper provides support to a variety of ferromagnetic payloads

The presented UAV platform is envisioned as a general-purpose payload delivery vehicle capable of executing autonomous flight missions in complex environments. A magnetic gripper is attached to the bottom of the frame. This allows the UAV to carry a useful payload such as a fire extinguisher ball. Furthermore, to enhance the environmental awareness capabilities, an Intel D435 RGB-D camera is mounted on the front of its frame.

2.2 Low-Level Control

The main objective of the low-level control is to reliably stabilize the vehicle during flight, both in terms of attitude and altitude. This is done in three steps: filtering setpoints, control loop, and thrust mixing. Thrust mixing refers to the distribution of the calculated controller outputs among the available rotors.

The proposed platform uses the controllers included in the Ardupilot - Copter firmware. The structure of the attitude controller, shown in Fig. 4, follows the form of a cascading PID controller with a single angle feed-forward term. An interesting feature is that the first proportional term of the angle controller has a nonlinear square root behavior for large angular errors. The altitude controller has a similar PID cascade structure.

The low-level controller has several different modes of operation both for attitude and altitude control. The three ways to control attitude are shown in Table 1. The first and last rows correspond to the roll, pitch, yaw rate, and complete rate control, respectively. These modes are typically used when an external type of state estimation is used, such as Optitrack Motion Capture (MOCAP) or simultaneous localization and mapping (SLAM). When using an external localization source that is not connected to the autopilot's internal localization, it is advantageous to use commands that refer to the drone's body reference point. On the other hand, if the autopilot's internal estimation is used, the preferred mode is the angle control shown in the second row of Table 1.

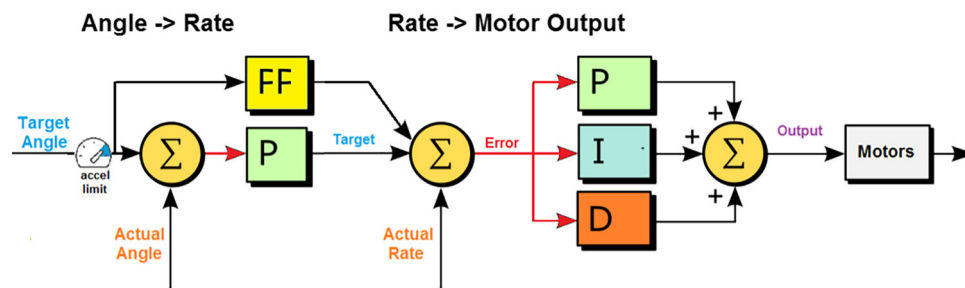


Fig. 4 This figure shows the Ardupilot - Copter attitude control scheme as presented in the Ardupilot codebase documentation⁷. The structure of the attitude controller is simplified based on the selected typemask, as

outlined in Table 1. If rate targets are used, only the inner loop is utilised; whereas, when angle targets are employed, both loops are engaged

As for the altitude control, there are two types of thrust interpretations, both scaled to the $[0, 1]$ range. By default, the thrust control input is interpreted as a climb rate, i.e. a vehicle ascends and descends at prescribed maximum speeds when the value is 0 and 1, respectively. Meanwhile, by setting the appropriate firmware parameter (GUID_OPTIONS=8), the controller interprets "thrust-as-thrust". In other words, the value of the hover thrust now depends on both the UAV mass and the battery voltage.

2.3 Onboard Control

The onboard controller is responsible for tracking the commanded trajectory, which consists of position, velocity, acceleration, and heading. In general, the controller differentiates the current trajectory point with the latest vehicle state obtained from the estimation module and produces control inputs based on the state error. The control inputs are sent via the Mavros topic `mavros/setpoint_raw/attitude` according to the guidelines explained in Section 2.2.

Table 1 This table shows all possible combinations of attitude control with the Ardupilot - Copter firmware

Attitude Target Typemask	Roll Control	Pitch Control	Yaw Control
3	Angle (body)	Angle (body)	Rate (body)
7	Angle (ENU)	Angle (ENU)	Angle (ENU)
128	Rate (body)	Rate (body)	Rate (body)

Changing the value of the attitude target typemask, a special MAVLink message supplied with each control input message changes the way the corresponding control input values are interpreted. For example, if the type mask indicates that the roll rate is used, the message field `body_rate.x` should be filled in. Body and ENU represent the body and East-North-Up angle (rate) frames of reference respectively

The main package used for onboard position control of the proposed UAV platform is `uav_ros_control`⁸. Here, a commonly used position control method, resembling a cascading PID structure with feed-forward elements similar to Fig. 4, can be found. Additionally, a standard model predictive controller is implemented, which can prove favorable when faced with noisy and unreliable state estimates.

As for the control methods for nonstandard UAVs, with additional sources of actuation such as moving masses or tilting propellers, a nonlinear geometric approach exploiting the variable center of gravity property[20] is included.

2.4 Trajectory Generation

The task of the trajectory generator is to produce a smooth trajectory across all vehicle states, based on the desired path. The generated trajectory is sampled and sent consecutively to the onboard control.

The main method for generating trajectories used on the presented UAV platform is the Time Optimal Path Parameterization by Reachability Analysis (TOPP-RA)[21] algorithm found inside the `topp_ros`⁹ package. It uses a constrained numerical integration approach that operates on the bang-bang principle of the actuator control value, which is acceleration in the case of the aerial vehicle.

An additional method used, inspired by the research conducted in [22], is a predictive, model-based trajectory generation method located in the `uav_ros_tracker`¹⁰ package. Given the desired path, it uses a model-predictive approach

⁸ Control algorithms used by the 'uav_ros_stack' package - https://github.com/larics/uav_ros_control

⁹ A ROS interface to TOPP and TOPP-RA - https://github.com/larics/topp_ros

¹⁰ The main trajectory generator of the 'uav_ros_stack' package - https://github.com/larics/uav_ros_tracker

to generate feasible trajectory points wrt. the model dynamics and given constraints.

2.5 State Estimation

The state estimation module is possibly one of the most important pipeline components for autonomous operations. It is responsible for providing a steady stream of UAV state estimates, i.e. position, velocity, acceleration, and heading. A general state estimation module usually uses a variant of the Kalman filter with IMU as the baseline measurement and additional sources such as barometer, GPS, visual SLAM, LiDAR slam, ultra wide-band (UWB) positioning, etc. The quality of the state estimate, which implies a low level of noise and time delay, strongly correlates with the flight stability of the UAV. Therefore, to obtain the best state estimate it is important to know the sensor characteristics and behavior in different environments. For example, a GPS does not work well indoors, visual SLAM usually does not perform well in changing light conditions, LiDAR SLAM performs poorly in featureless environments, etc.

The UAV platform presented includes several variants of state estimation. The most reliable source of localization measurements is a motion capture system. For the purposes of this paper, a UAV with reflective markers is tracked by the Optitrack system utilizing the *ros_vrpn_client*¹¹. A Luenberger state observer is included to provide accurate and stable model estimates.

Sometimes it is simply not feasible to set up an indoor motion capture system due to the size of the arena and the obstacles within it. Therefore, an alternative is to use multiple sources of measurement with a sensor fusion framework. This is done with the *sensor_fusion*¹² package [23] and successfully deployed in GPS-denied environments. To achieve this, an error-state Kalman filter with position and orientation drift estimation is implemented and various sensors such as visual and LiDAR SLAM and UWP position are used as correction measurements.

Finally, for outdoor competition, the safest localization option is GPS. Directly connected to the autopilot, the UAV state estimate can be easily obtained by listening to the Mavros topic `mavros/global_position/local`.

2.6 Global Path Planning

In general, the goal of global path planning is to find a feasible, collision-free path based on the vehicle's initial and target states given a map of the environment. Therefore, the

input to such a system consists of an occupancy grid of the environment and a set of waypoints, while the output is a piecewise straight line connecting the provided waypoints.

The package used for this purpose is *larics_motion_planning*.¹³ It uses the Rapidly-exploring Random Tree (RRT)[24] method to generate a collision-free path. The RRT algorithm starts from vehicle's initial state and performs a random sampling in multiple directions. Valid states from the random sample are added to the tree and the same search is performed again for the new nodes. This way, the tree grows in all directions trying to reach the goal which makes it suitable for exploring large environments. The RRT based path is then parameterized using the TOPP-RA algorithm, yielding a trajectory suitable for the UAV to execute.

For more information about potential use cases for the path planning algorithm, the reader is referred to parabolic air-drop experiments presented in [25] and its model-based path planning variant from [26].

2.7 Mapping

The vehicle needs a way to understand its environment, which requires the introduction of some kind of mapping into the system. A map is usually represented as an occupancy grid, meaning that the environment is divided into fixed-size cubes, also known as voxels, that mark space as either occupied or unoccupied. This leads to performance and memory issues for larger environments. Therefore, an efficient algorithm for 3D space decomposition, called OctoMap[27], is employed. A visual example of an OctoMap is presented in Fig. 5. This format of environment representation is well suited for collision checking and subsequent path planning. Another important aspect to consider is localization. Assuming perfect localization, the mapping task is straightforward if a sensor contains depth information. However, poor localization may require a SLAM approach.

Suitable mapping sensors mounted on the presented UAV platform include only the Intel Realsense D435 depth camera. However, a Light Detection and Ranging sensor (LiDAR) is also very popular in UAV applications and could be mounted in a different scenario. We opted for a simple depth camera, to lower the cost of the platform. The Cartographer[28] SLAM algorithm is tightly integrated with the described software modules using the *uav_ros_cartographer*¹⁴ package. It is useful both for path planning and state estimation purposes [29].

¹¹ A ROS VRPN interface - https://github.com/ethz-asl/ros_vrpn_client

¹² A multi-sensor error-state EKF implementation for UAV localization - https://github.com/larics/sensor_fusion

¹³ An RRT-based motion planning module - https://github.com/larics/larics_motion_planning

¹⁴ This repository integrates Cartographer with the 'uav_ros_stack' packages - https://github.com/larics/uav_ros_cartographer

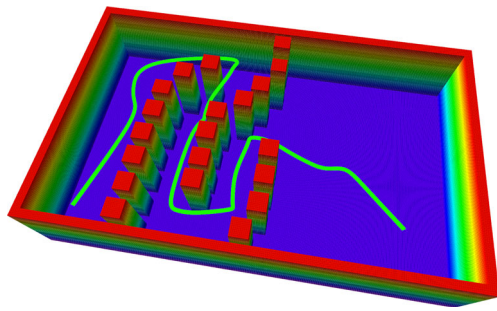


Fig. 5 This figure shows a collision-free trajectory in a maze-like environment planned using the RRT path planning and TOPP-RA trajectory generation from the *larics_motion_planning* package

2.8 Gazebo Simulation

Up to this point, the software modules described are all a part of or closely related to the *uav_ros_stack*[19] package. These modules are capable of running natively on the computer onboard the UAV platform. In this section, it is shown how exactly these modules are integrated into the Gazebo simulation environment providing a software-in-the-loop framework. The obvious advantage is a seamless transition of new features from the simulation to the experimental validation.

All of the software modules as well as the required simulation components are integrated under the *uav_ros_simulation*[30] package. The outline of the simulation pipeline is shown in Fig. 6.

The core of the Gazebo UAV model is the rotor model plugin from the *rotors_simulator*[17] package, which real-

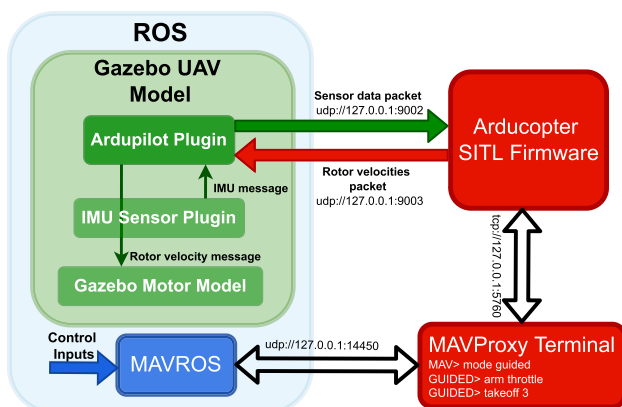


Fig. 6 This figure shows the pipeline connecting the Arducopter firmware to the Gazebo simulator and ROS, enabling the software-in-the-loop simulations. On one end there is a Gazebo UAV model that can control its motor velocities, while on the other end is the connection to the pipeline shown in Fig. 2 via the *Control Inputs*. In both the simulation and real-world deployment, *Mavros* provides a generalized interface to the Gazebo model or the actual vehicle respectively. This allows a seamless transition of software modules from the simulation to the real world

istically simulates the forces and moments generated by the propellers. The connection to the Ardupilot firmware, running the Copter- SITL binaries, is established via UDP. The firmware needs to receive all of the available sensor data such as IMU, position and velocity measurements and in order to output the target rotor velocities. Finally, the MAVProxy terminal, which is a fully-fledged command-line ground control station (GCS), relays the information stream to and from the Mavros node. All of the necessary components for running the Ardupilot Gazebo SITL are found in the *ardupilot_gazebo*¹⁵ package.

3 Competition Scenario: Firefighting UAVs

In this section, a use case for the proposed UAV platform is showcased in the form of a competition. First, the competition tasks and their original motivation are stated. Furthermore, each task is described in detail and the scoring scheme is given. Finally, a discussion about the challenge setup, the experiences, and the results of the simulation qualifiers along with the Dubrovnik arena finals is presented.

3.1 Competition Description

The competition is geared towards students and provides them with a unique opportunity to test their knowledge and expertise against their peers from all around the world. The initial qualifying stages were conducted in the Gazebo simulation environment while the finals were held onsite at the ICUAS'22 conference in Dubrovnik, Croatia.

The competition is based on the challenges faced by firefighting UAVs. In the envisioned scenario of a burning building, UAVs can be deployed to extinguish the fires by throwing a fire extinguisher ball from a safe distance at the fire. Such a scenario can be divided into three separate tasks: exploration, target detection, and precision payload delivery. With respect to the competition layout shown in Fig. 7, exploration takes place in *Zone 2*, while target detection and precision payload delivery are performed in *Zone 3*. The layout dimensions are provided to the competitors, both for the simulation and the onsite arena. To successfully complete the competition challenge, participants must complete each task sequentially and autonomously during a single flight. The run starts as soon as the UAV takes off and ends either after a predetermined timeout duration or when the payload is released from the magnetic gripper.

Exploration The first task evaluates the UAV's ability to navigate through a set of static obstacles. To successfully

¹⁵ A repository for ArduPilot & Gazebo Software In Loop Simulation Interfaces, Models - https://github.com/larics/ardupilot_gazebo

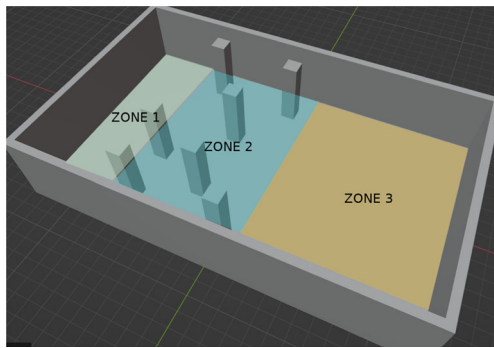


Fig. 7 This figure shows a generalized layout model of the competition scenario. *Zone 1* is the designated takeoff area, free of any obstacles. *Zone 2* includes an obstacle-cluttered environment that evaluates exploration and obstacle avoidance solutions. Finally, *Zone 3* includes an *ArUco* marker at an unknown position. This part tests the target detection and precision payload delivery segments of the competitor's solution

complete this task, the UAV needs to autonomously travel all the way from *Zone 1* to *Zone 3*, as shown in Fig. 7, without colliding with the environment.

There are two levels of difficulty for this task. Participants can choose to use a pre-made map of the arena environment and plan a collision-free path based on the given map. Alternatively, they can use the sensor data available on the drone, namely the RGB-D camera, to navigate through the obstacles. Regardless of the total point count, those who choose the former option are ranked below the participants who chose the latter.

Target Detection In this task, an *ArUco* is placed somewhere in *Zone 3*. To successfully complete this task, the UAV needs to find and reconstruct the position of the *ArUco* marker. The reconstructed position, corresponding to the center of the marker, must be published to a predefined ROS topic to evaluate the competitors' target detection solution.

Precision Payload Delivery Once the UAV finds the *ArUco* marker, it can begin the execution of the final task. The goal of this task is to launch the payload and hit the located *ArUco* marker. It is important to note that the orientation of the marker can be anywhere from horizontal to vertical. This means that a simple airdrop solution is not sufficient to accomplish this task. Instead, a series of trajectory points should be generated for the UAV in such a way that when the magnetic gripper releases the payload, it flies in a parabolic trajectory and hits the desired target.

3.2 Scoring

Teams receive points only if the run is considered valid. The simulation score is calculated as an average of several runs,

while in the on-site final the team with the highest score in a single run wins. The scoring scheme is given as follows:

$$\text{Reconstructed target position accuracy:} \quad (1)$$

$$P1 = 25 \exp(-2\|\mathbf{p}_{\text{detected}} - \mathbf{p}_{\text{target}}\|),$$

$$\text{Accuracy of the ball impact wrt. the target:} \quad (2)$$

$$P2 = 35 \exp(-0.5\|\mathbf{p}_{\text{impact}} - \mathbf{p}_{\text{target}}\|),$$

$$\text{Time required to complete the run:} \quad (3)$$

$$P3 = 40 \exp\left(-\frac{t}{100}\right).$$

Penalties and Disqualifying Behaviour The competitors do not receive points for the run if:

- The solution code submitted is unable to be executed either on the evaluation machine or the UAV platform,
- The UAV crashes after collision with obstacles at any point during the run,
- The run exceeds the specified timeout duration,
- The ball is released before reaching *Zone 3*.

In case the UAV touches the obstacles but continues to complete the task, teams are penalized by deducting 15% of total points achieved in that run for each contact with the obstacle.

3.3 Gazebo Simulation Qualifiers

During the initial stage of the the simulation qualifiers, participating teams received the rulebook and the *icuas22_competition* [31] repository where they found installation and simulation startup instructions. The competition backend includes the *uav_ros_simulation* [30], which participants can either install natively, build with Docker or download a pre-built Docker image from DockerHub¹⁶. Participants are able to raise issues or start a discussion directly on the *icuas22_competition* GitHub repository. Furthermore, Tmux / Tmuxinator based routine is provided to easily run both the competition simulation and the teams' solution. The simulation includes a Gazebo maze-like environment and a single UAV with a mounted magnetic gripper, as shown in Fig. 8. After starting the simulation, the UAV takes off and the magnetic payload is spawned under the UAV, signaling the beginning of the challenge.

As for the solution, there are no software limitations. Teams can use any number of existing algorithm implementations or create their own. Each node used for the solution is expected

¹⁶ Pre-built images for various Ubuntu Linux versions of the 'uav_ros_simulation' package - https://hub.docker.com/r/lmark1/uav_ros_simulation

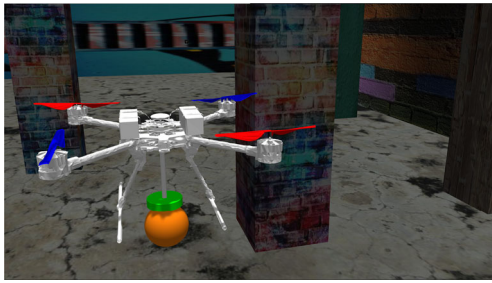


Fig. 8 This figure shows the layout for the simulation stage of the competition. This includes an Arducopter SITL Gazebo simulation of the proposed aerial vehicle platform in a maze-like environment with a hidden *ArUco* marker. The UAV is equipped with similar sensors and payload as shown in Fig. 3

to run autonomously, without external support, using the Tmux environment mentioned earlier. The only requirement is that teams combine all their packages into a single ZIP file or alternatively provide a working Dockerfile.

3.4 Dubrovnik Arena Finals

The finals were held in Dubrovnik, Croatia, during the ICUAS'22 conference. There were five teams that qualified to the finals. The only requirement for the teams was to bring their own personal laptop in order to improve and modify their solution during the competition, while everything else was provided by the organizing committee. The complete setup of the competition arena is shown in Fig. 9. The first part of the arena consists of obstacles such as vertical poles, balloons and boxes. The second part of the arena has an *ArUco* marker over the box opening, which represents the payload dropoff area. The Optitrack motion capture cameras were placed at the edges of the arena to provide a consistent localization source for the UAV.



Fig. 9 This figure shows one of the competition runs of the firefighting challenge. There is a safety margin as well as a net placed around the designed flight area to provide a safe spectator experience

The finals were organized over three days:

- Day 1 - Integration workshop: Teams were invited to the competition arena to meet the organizing committee, judges and to familiarize themselves with the arena setup, the UAV, and the procedure of launching their code on the UAV. Additionally, the teams' solutions were checked for build errors and whether they could be deployed on the UAV without issues.
- Day 2 - Practice day: Each of the five teams was allocated two 30 minutes long slots to test their solution.
- Day 3 - Competition day: Similar to the practice day, each team was granted two slots of 30 minutes to perform as many runs as they could fit in the slot.

Similar to the simulation phase, the UAV's onboard computer came prepared with both native and Docker installations of the *uav_ros_stack*[19]. The competitors were able to either install their solution natively, in separate workspaces, or use a containerized approach. The latter approach is preferred because it minimizes the risk of dependency collisions and makes it easier for competitors to deploy their solutions. The only restriction imposed on the competitors was the use of low-level controller, which they weren't allowed to change in terms of structure, but they were allowed to calculate and use different parameters.

Before each run, participants had 5 minutes to install or update their existing solution on the UAV's onboard computer. Afterwards, they spent the remaining 25 minutes trying to complete the entire challenge in a single run. Live stream of the entire competition can be seen at [1].

To ensure UAV safety, additional collision checks were imposed on the generated trajectory inputs against an OctoMap model of the competition layout. Fortunately, no UAVs were harmed in the making of this competition.

Scoring in the finals The scoring for the finals was adapted from the simulation phase, but still focused on the target position reconstruction and ball launch accuracy:

- Target reconstruction error less than 0.5 m: 25 points
- Launching the ball on the top surface of the target box (shaded blue in Fig. 1): 25 points
- Launching the ball on the white border surrounding the target tag (shaded green in Fig. 1): 50 points
- Hitting the hole in the box that was placed behind the tag: 75 points

The final score for the team was the score achieved in their best run, and time to finish the run was used as a tie breaker.

4 Competition Timeline And Overall Reach

The competition was announced on the ICUAS'22 website¹⁷ on November 23rd 2021. Interested teams were asked to fill a form to pre-register for the competition and officially declare their interest in competing. In total, more than 50 teams from 22 countries declared their interest.

The competition kicked off on February 1st 2022 (simultaneous with ICUAS'22 paper submission deadline) with the release of the software, several practice Gazebo worlds, and a rulebook. Starting from April 1st, teams were allowed to submit their solutions for evaluation. The evaluation was performed in a Gazebo world not shared with the teams. Results from these evaluations were shared with all teams via a public web page, but the teams were kept anonymous, providing the submitting team with valuable feedback but also providing a target for other teams. Along with the score for the run, teams were provided with a detailed log of timestamped events in the run. The timeline of team interest, registration, and solution submissions is shown in Fig. 10. In total, 62 valid solutions were evaluated in the simulation phase. Teams were also instructed to complete their registration by providing proofs of student status, which was completed by 16 teams consisting of 125 people throughout April. Of those 16 fully registered teams, only one team did not submit a valid solution by the end of simulation phase. The simulation phase ended on May 10th 2022, with 15 teams submitting valid solutions that were then evaluated. Simultaneously, the Gazebo worlds and automated scripts used for evaluation were released to the teams to validate achieved results on their own platform. No complaints were received from the teams.

The finals were held at the ICUAS'22 conference venue in Dubrovnik, Croatia. Five teams successfully qualified for the finals and received invitations to participate onsite. Consequently, all of these finalist teams were present at the conference, accounting for a total of 14 individuals in attendance. Competition arena was visited by more than 200 ICUAS participants, and live streams of the practice and the competition have been viewed more than 400 times. The competition Github repository [31] has been cloned more than 200 times and visited by more than 1000 unique visitors.

5 Conclusion

This paper attempts to distinguish and standardize both the hardware and software modules of an aerial vehicle capable of autonomous operations. Having a modular approach in mind, all of the requirements are clearly stated so that each component can readily be replaced with likewise pieces.

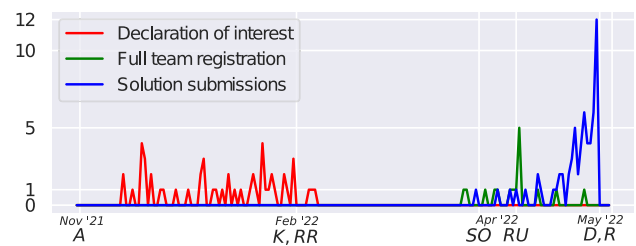


Fig. 10 The timeline of the ICUAS'22 UAV Competition simulation phase. Starting from November 23rd 2021 when the competition was announced (labeled with A), most of the interest within teams was generated up until the competition kickoff (label K) on February 1st 2022. Following the competition kickoff (K) and rulebook release (RR), and with passage of ICUAS'22 paper deadline, no further interest was generated and pre-registration phase was closed on March 15th. On April 1st 2022, teams were allowed to submit their solution for evaluation (label SO), which prompted a few teams to submit their preliminary work. Following the rulebook update on April 13th (label RU), which finalized the scoring scheme, the number of solutions submitted rises towards the final deadline on May 10th (label D). Results of the simulation stage and finalists were announced on May 14th (label R)

Furthermore, a concrete example of such an aerial platform, with all of its components outlined in a standardized manner, is presented. A complementary Ardupilot SITL Gazebo simulation structure is introduced to provide a seamless transition from a simulation environment to the real-world deployment.

Finally, a use case of the presented UAV platform is demonstrated in the form of a firefighting UAV competition organized as a part of ICUAS'22. Leveraging deployment tools such as Docker, the platform proved accessible to the competitors both in a simulation environment and the onsite challenge arena. The scoring scheme is set up in such a way to facilitate a fair and transparent environment, where the competitors are able to easily evaluate their own solutions.

In total, 125 people across 16 teams attempted the Gazebo simulation qualifiers, while 5 teams qualified for the finals. Each of the five teams managed to successfully deploy their solution using the presented UAV platform and 3 teams successfully completed the entire challenge autonomously in a single run.

Tying the competition to the ICUAS conference has been a major success as it attracted a large number of teams. As evidenced by the interest graph, without the conference promoting the call for papers that briefly presents the competition, it is generally difficult to generate additional interest in the competition without large financial incentives. The most notable drawback of this approach is the amount of logistics required to construct a safe arena that enables fair, transparent and fast scoring at the conference venue. To further increase the safety, but also the availability of the aerial platform, we aim to replace the large UAV used in ICUAS'22 with a smaller frame. This, coupled with the ongoing development and refinement of the UAV platform, will also open

¹⁷ http://www.uasconferences.com/2022_icuas/uav-competition/

up different scenarios for future competitions. The scoring at the finals was also difficult to perform since only small parts of it were automated and benchmarked. Therefore, our aim for the next competition at ICUAS'23 is to join forces with the METRICS project to bring more benchmarks from the simulation phase to the competition arena at the conference venue.

Acknowledgements This paper presents results of development that has been carried out as a part of the scientific project Autonomous System for Assessment and Prediction of infrastructure integrity (ASAP) financed by the European Union through the European Regional Development Fund - The Competitiveness and Cohesion Operational Programme (KK.01.1.1.04.0041) and as a part of the scientific project Development of a system of unmanned aerial vehicles (UAVs) controlled in virtual environments (VIRTUALUAV) financed by the European Union through the European Regional Development Fund (KK.01.2.1.02.0197).

The authors would like to thank all of LARICS staff, ICUAS and sponsors for their help and support in carrying out the competition at ICUAS'22 in Dubrovnik

Author Contributions All of the authors contributed equally to the realization of this work

Funding As stated in the Acknowledgements section, this research is a part of the scientific project Autonomous System for Assessment and Prediction of infrastructure integrity (ASAP) financed by the European Union through the European Regional Development Fund - The Competitiveness and Cohesion Operational Programme (KK.01.1.1.04.0041). Furthermore, this research is a part of the scientific project Development of a system of unmanned aerial vehicles (UAVs) controlled in virtual environments (VIRTUALUAV) financed by the European Union through the European Regional Development Fund (KK.01.2.1.02.0197)

Availability of data and materials Not applicable

Code availability The code is open-source and available at the Github repositories [19], [30] and [31]

Declarations

Conflicts of interest Not applicable

Ethics approval Not applicable

Consent to participate Not applicable

Consent for publication Not applicable

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copy-

right holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. UNIZG-FER, Larics.: ICUAS'22 UAV Competition Finals. <https://www.youtube.com/watch?v=S2c0bQQ8U&list=PLC0C6uwoEQ8Z1GbMr8n7kFB442iyBp20g> Accessed: 30 Nov 2022
2. Amigoni, F., Bastianelli, E., Berghofer, J., Bonarini, A., Fontana, G., Hochgeschwender, N., et al.: Competitions for benchmarking: Task and functionality scoring complete performance assessment. *IEEE Rob Autom Mag* **22**(3), 53–61 (2015). <https://doi.org/10.1109/MRA.2015.2448871>
3. Kitano, H., Asada, M., Kuniyoshi, Y., Noda, I., Osawa, E.: RoboCup: the Robot World Cup Initiative. In: *Proc Int Conf Auton Agent* 1998 04 <https://doi.org/10.1145/267658.267738>
4. Siciliano, B., Caccavale, F., Zwicker, E., Achtelik, M., Mansard, N., Borst, C., et al.: EuRoC - The challenge initiative for european robotics. In: *ISR/Robotik 2014; 41st Int Symp Robot p* 1–7 (2014)
5. Winfield, AFT., Palau Franco, M., Brueggemann, B., Castro, A., Ferri, G., Ferreira, F., et al.: euRathlon and ERL Emergency: A multi-domain multi-robot grand challenge for search and rescue robots. In: Ollero, A., Sanfeliu, A., Montano, L., Lau, N., Cardeira, C.: (eds) *ROBOT 2017: Third Iberian Robotics Conference*. Cham: Springer International Publishing p 263–271 (2018)
6. METRICS: Metrological evaluation and testing of robots in international competitions. <https://metricsproject.eu/> Accessed 30 Nov 2022
7. DARPA Subterranean Challenge. <https://www.subtchallenge.com/> Accessed 30 Nov 2022
8. ERL - European Robotics League. <https://eu-robotics.net/eurobotics/activities/european-robotics-league/> Accessed 30 Nov 2022
9. MBZIRC. <https://www.mbzirc.com/> Accessed 30 Nov 2022
10. Aerial robotics control and perception challenge. <https://www.med-control.org/med2018/?pageid=602> Accessed 30 Nov 2022
11. Ferri, G., Ferreira, F., Faggiani, A., Fabbri, T.: From ERL to RAMI: Expanding marine robotics competitions through virtual events. In: *OCEANS 2021: San Diego-Porto* p 1–8 (2021)
12. Laue, T., Moos, A., Götsch P.: Let your robot go - challenges of a decentralized remote robot competition. In: *2021 European Conf Mobile Robots (ECMR)* p 1–7 (2021)
13. Cabrita, G., Madhavan, R., Marques, L.: A framework for remote field robotics competitions. In: *IEEE Int Conf Auton Robot Syst Competitions* p 192–197 (2015)
14. Robosoft competition. (2022) <http://web.archive.org/web/20211026125532/>, https://softroboticsconference.org/wp-content/uploads/2021/10/2022_Robosoft_Call_for_Competition.pdf. Accessed 30 Nov 2022
15. MATAMOROS, M., SEIB, V., PAULUS, D.: Trends, challenges and adopted strategies in RoboCup@Home. In: *IEEE International Conference on Autonomous Robot Systems and Competitions (ICARSC)* p 1–6 (2019)
16. Baca, T., Petrlik, M., Vrba, M., Spurny, V., Penicka, R., Hert, D., et al.: The MRS UAV System: Pushing the Frontiers of Reproducible Research. In: *Real-world Deployment, and Education with Autonomous Unmanned Aerial Vehicles*. *J Intell Robot Sys* **102**(1). Apr 2021 <https://doi.org/10.1007/s10846-021-01383-5>
17. Furrer, F., Burri, M., Achtelik, M., Siegwart, R.: RotorS—A Modular Gazebo MAV Simulator Framework. In: Koubaa, A. (ed.) *Robot Operating System (ROS): The Complete Reference*. Cham: Springer International Publishing (Volume 1) p 595–625 (2016) Available from: <http://dx.doi.org/10.1007/978-3-319-26054-923>

18. Sanchez-Lopez, J.L., Suárez Fernández, R.A., Bavlé, H., Sampedro, C., Molina, M., Pestana, J., et al.: AEROSTACK: An architecture and open-source software framework for aerial robotics. In: 2016 Int Conf Unmanned Aircr Sys (ICUAS) p 332–341 (2016)
19. Markovic, L., et al.: uav ros stack - A Stack of ROS Packages for Ardupilot Compatible Unmanned Aerial Vehicles. <https://github.com/larics/uavrostack> Accessed 30 Nov 2022
20. Markovic, L., Ivanovic, A., Car, M., Orsag, M., Bogdan, S.: Geometric tracking control of aerial robots based on centroid vectoring p 2701–2706 (2019)
21. Pham, H., Pham, Q.C.: A new approach to time-optimal path parameterization based on reachability analysis. *IEEE Trans Robotics* **0634**, 645–659 (2018) <https://doi.org/10.1109/TRO.2018.2819195>
22. Baca, T., Hert, D., Loianno, G., Saska, M., Kumar, V.: Model predictive trajectory tracking and collision avoidance for reliable outdoor deployment of unmanned aerial vehicles. In: IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) p 6753–6760 (2018)
23. Marković, L., Kovač, M., Milijas, R., Car, M., Bogdan, S.: Error state extended kalman filter multi-sensor fusion for unmanned aerial vehicle localization in GPS and magnetometer denied indoor environments. In: 2022 Int Conf Unmanned Aircr Syst (ICUAS) p 184–190 (2022)
24. LaValle, S.M.: Rapidly-exploring random trees: A new tool for path planning. The Annual Research Report (1998)
25. Ivanovic, A., Orsag, M.: Parabolic airdrop trajectory planning for multirotor unmanned aerial vehicles. *IEEE* **10**, 36907–36923 (2022). <https://doi.org/10.1109/ACCESS.2022.3164434>
26. Ivanovic, A., Markovic, L., Car, M., Duvnjak, I., Orsag, M.: Towards autonomous bridge inspection: Sensor mounting using aerial manipulators. *Applied Sciences* **11**(18) (2021) <https://doi.org/10.3390/app11188279>
27. Hornung, A., Wurm, K.M., Bennewitz, M., Stachniss, C., Burgard, W.: OctoMap: An efficient probabilistic 3d mapping framework based on octrees. *Autonomous Robots* (2013) Software available at <https://octomap.github.io>. <https://doi.org/10.1007/s10514-012-9321-0>
28. Hess, W., Kohler, D., Rapp, H., Andor, D.: Real-time loop closure in 2D LIDAR SLAM. In: 2016 IEEE Int Conf Robotics and Automation (ICRA) p 1271–1278 (2016)
29. Milijas, R., Markovic, L., Ivanovic, A., Petric, F., Bogdan, S.: A Comparison of LiDAR-based SLAM systems for control of unmanned aerial vehicles. In: 2021 Int Conf on Unmanned Aircraft Systems (ICUAS) p 1148–1154 (2021)
30. Markovic, L., et al.: uav ros simulation - A collection of ROS packages for Gazebo simulations of Ardupilot UAV platforms. Accessed 2022-11-29. https://github.com/larics/uavros_simulation
31. Markovic L, Petric F, Ivanovic A, Car M.: The main repository for the ICUAS 2022 UAV competition. <https://github.com/larics/icuas22competition> Accessed 30 Nov 2022

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Lovro Markovic received his M.S.E.E. in 2018. and B.S.E.E. in 2016. at the University of Zagreb, Croatia. He is currently employed at the Faculty of Electrical Engineering and Computing, University of Zagreb, as a research associate and a Ph.D. student where he participated in numerous international and national scientific projects. His main areas of interest include aerial robotics, geometric control theory and autonomous systems.

Frano Petric received his Ph.D. from the University of Zagreb Faculty of Electrical Engineering and Computing (UNIZG-FER) in March of 2018. He received the Rector's Award in 2011 for his work on control of walking robots. In October of 2012, he started working in the Laboratory for Robotics and Intelligent Control Systems (LARICS) at the Department of Control and Computer Engineering at the same faculty, where he has since been involved in several international and national projects. Currently, he is employed as a postdoctoral researcher. His scientific focus is on social and assistive robotics and robots operating under uncertainty. His other scientific interests include autonomous systems, walking robots, computer science, and computer vision. He is also involved in science popularization activities of the faculty.

Antun Ivanovic received his M.S.E.E. in 2015 and B.S.E.E. in 2013 at the University of Zagreb, Croatia. He is currently a Ph.D. student and is employed as a researcher at the Laboratory for Robotics and Intelligent Control Systems within the University of Zagreb Faculty of Electrical Engineering and Computing (UNIZG-FER). He graduated and received his master's degree in 2015 at this same institution and after that joined LARICS later that year. During his undergraduate studies, he received a Rector's award for the work entitled "Augmented human machine interface for aerial manipulators". His research interests are robotics, unmanned aerial vehicles, aerial manipulation, and motion planning. As a Ph.D. student, he participated in numerous international and national research projects. In 2018, he was a visiting researcher at United States Military Academy West Point, USA, where he collaborated on work related to aerial-ground cooperative manipulation.

Jurica Goricanec received the B.S.E.E. in 2011. and M.S.E.E. in 2013. degrees from the University of Zagreb, Croatia. He is currently a Ph.D. student and Research Associate in the Laboratory for Robotics and Intelligent Control Systems (LARICS) within the University of Zagreb Faculty of Electrical Engineering and Computing (UNIZG-FER). He received the Rector's Award in 2012 for his work on development of self-rotating aerial vehicle. After graduation, he worked on development and usage of unmanned aerial vehicles in archaeology excavations, infrastructure surveys and terrain mapping.

Marko Car is currently employed at the Faculty of Electrical Engineering and Computing, University of Zagreb, as a research associate and a Ph.D. student. His main areas of interest are aerial robotics, control theory and autonomous systems. In his undergraduate years he received University of Zagreb Rector award for the best student projects in 2014.

Matko Orsag received his Ph.D. in 2015, M.S.E.E. in 2010 and B.S.E.E. in 2008 at the University of Zagreb, Croatia. He is currently an assistant professor at the University of Zagreb Faculty of Electrical Engineering and Computing (UNIZG-FER). As a researcher, he participated in several national and international research projects in the field of robotics, control, and automation. In 2011/2012, he worked as a visiting researcher at the Drexel University, Philadelphia, USA as a recipient of the Fulbright exchange grant. He authored and coauthored over 30 scientific and professional papers, including journal and conference papers, as well as a monograph and a book chapter in the field of unmanned aerial systems and robotics. His main areas of interest are autonomous systems, robotics and intelligent control systems.

Stjepan Bogdan received his Ph.D.E.E. in 1999, M.S.E.E. in 1993 and B.S.E.E. in 1990 at the University of Zagreb, Croatia. He is a Full Professor at the Laboratory for Robotics and Intelligent Control Systems (LARICS), Department on Control and Computer Engineering, University of Zagreb Faculty of Electrical Engineering and Computing (UNIZG-FER). His main areas of interest are autonomous systems, aerial robotics, multi-agent systems, intelligent control systems, bio-inspired systems and discrete event systems. He spent one year as Fulbright researcher at the Automation and Robotics Research Institute, Arlington, USA, in Prof. Frank Lewis' lab. He is a coauthor of four books and numerous papers published in journals and proceedings.